

PP0 AGENT MODEL EVALUATION PLAN (ASCII VERSION)

Setup / facts (as given)

- You have 15 total instances in a fixed pool.
- You trained 5 separate PP0 models.
- For each model:
 - Training set: 12 randomly chosen instances (from the 15).
 - Test set: the remaining 3 instances (held out from training for that model).
 - Evaluation on test: each of the 3 test instances is run with 20 random seeds.

Goal

1. Compare the 5 PP0 models on held-out generalization.
2. Measure variance across seeds.
3. Optionally compare vs baselines (CP-SAT, ALNS variants, heuristics) if available.
4. Diagnose policy behavior (action usage, state visitation, failure modes, learning stability)

PART I - DATA INVENTORY AND CONSISTENCY CHECKS

Task 1 - Freeze splits and metadata (critical)

For each model m in $\{1..5\}$ write down and store:

1. Train instance IDs (12)
2. Test instance IDs (3)
3. Training budget: timesteps, updates, epochs, batch size, gamma, lambda, clip range, entropy
4. Reward definition and scaling (exact formula in code)
5. Observation and action definitions (dimensions, normalization, clipping)
6. Any domain randomization used during training
7. Environment version / commit hash

Deliverable:

A "model_card.csv" (or JSON) with one row per model capturing all items above.

Task 2 - Collect evaluation runs (test only)

Create a run-level table. One row per (model, test_instance, seed):

Fields (minimum):

1. model_id
2. test_instance_id
3. seed (1..20)
4. final_objective
5. best_objective_over_episode (if tracked)
6. episode_length (steps)
7. runtime_seconds (wall-clock)
8. termination_reason (time limit, solved, max steps, invalid action, etc.)
9. any_constraint_violation_metric (if your objective includes penalties, record raw violations)
10. optional: return (sum of rewards) and/or discounted return

Deliverable:

A single dataframe with $5 * 3 * 20 = 300$ rows.

Task 3 - Sanity checks on raw data

For each model and test instance:

1. Confirm all 20 seeds exist.
2. Confirm environment settings (time limits, step limits) are identical across models.
3. Confirm objective direction (minimization) is consistent.
4. Confirm no NaNs, no missing values.
5. Confirm termination_reason values are valid and consistent.

Deliverable:

A short "data_quality.md" checklist with pass/fail and notes.

=====

PART II - PRIMARY GENERALIZATION METRICS (PER TEST INSTANCE)

=====

Task 4 - Per-instance aggregation per model

For each model m and each of its 3 test instances i :

Compute from the 20 seeds:

1. `mean_objective(m,i)`
2. `median_objective(m,i)`
3. `best_objective(m,i)` = min over seeds
4. `worst_objective(m,i)` = max over seeds
5. `std_objective(m,i)`
6. `range_objective(m,i)` = worst - best
7. `quantiles_objective(m,i)`: q10, q25, q50, q75, q90

If you have feasibility/violations separated:

8. `mean_violations(m,i)`
9. `P(no_violations)(m,i)` if meaningful

Deliverable:

A per-model per-test-instance table with $5 * 3 = 15$ rows.

Task 5 - Normalize objectives to a baseline (if you have one)

If you have CP-SAT objective $f_{CP}(i)$ per instance i , define for each run:

$$\text{gap}(m,i,s) = (f(m,i,s) - f_{CP}(i)) / \text{abs}(f_{CP}(i))$$

Then compute `mean_gap`, `median_gap`, `best_gap`, `std_gap` per (m,i) .

Deliverable:

Same as Task 4 but for gap metrics.

=====

PART III - MODEL COMPARISON ACROSS THE 5 MODELS

=====

Important structural issue:

Each model has different held-out test instances (because the 12/3 split is random per model). So you must decide what "comparison" means. Do both A and B below.

Task 6A - Comparison on each model's own held-out test set (within-model evaluation)

For each model m , define its held-out score as an aggregate over its 3 test instances:

Option 1 (mean-of-means):

$$\text{score_mean}(m) = \text{mean over } i \text{ in Test}(m) \text{ of } \text{mean_objective}(m,i)$$

Option 2 (median-of-means):

score_median(m) = median over i in Test(m) of mean_objective(m,i)

Option 3 (robust worst-case):

score_worstcase(m) = max over i in Test(m) of mean_objective(m,i)

Also compute corresponding std across the 3 test instances to reflect heterogeneity.

Deliverable:

One table with 5 rows (one per model) and the three scores above.

Notes:

- With only 3 test instances per model, inference is weak. Treat as descriptive.

Task 6B - Cross-model comparison on the union of test instances (recommended)

Create the set U = union of all test instances across the 5 models.

This is a subset of the 15 (possibly all 15 depending on splits).

For each instance i in U and each model m:

- If i was in Test(m): use the 20-seed evaluation results.
- If i was in Train(m): do NOT use training-time results. Instead:
 - 1) run an evaluation for that model on instance i with 20 seeds (preferred), or
 - 2) exclude (m,i) from cross-model tables if you cannot re-run.

This yields a rectangular comparison table across models on a common set of instances.

Deliverable:

A matrix: rows = instances in U, columns = models, entries = mean_objective over 20 seeds.

Task 7 - Ranking stability and dominance

On the chosen common evaluation set (Task 6B preferred):

1. For each instance i, rank models by mean_objective(m,i).
2. Count per model:
 - number of wins (rank 1)
 - top-2 count
 - average rank
3. Identify pairwise dominance:
model A dominates B if mean_objective(A,i) <= mean_objective(B,i) for most instances,
and strictly better on at least one, with a clear margin threshold if you want.

Deliverable:

Ranking and dominance summary tables.

=====

PART IV - STATISTICAL TESTING (WHERE IT IS VALID)

=====

Because test sets differ by model, only do hypothesis tests on comparisons that are paired

Task 8 - Paired tests on a common instance set

If you construct a common evaluation set U (Task 6B) with results for every (model, instance)

For any pair of models A and B, define per-instance differences:

`d_i = mean_objective(A,i) - mean_objective(B,i) for i in U`

Apply Wilcoxon signed-rank test on `{d_i}`.

Report:

- p-value
- median(`d_i`)
- mean(`d_i`)
- counts: `#better`, `#worse`, `#ties`

Deliverable:

A 5x5 pairwise comparison table (upper triangle) with p-values and effect sizes.

If the matrix is not complete (missing some `(m,i)`):

- either re-run to complete it, or
- perform tests on the intersection of instances evaluated for both models and record the

Task 9 - Multiple comparisons control

If you do all 10 pairwise comparisons among 5 models, decide and report:

1. No correction (exploratory only), or
2. Holm-Bonferroni correction, or
3. Benjamini-Hochberg FDR control.

Deliverable:

Same pairwise table but with adjusted significance decision.

=====

PART V - SEED VARIANCE AND RISK PROFILE

=====

Task 10 - Seed-level risk metrics per (model, instance)

For each `(m,i)` on test:

1. `std_objective(m,i)`
2. `CV(m,i) = std / mean` (use only if mean > 0 and scale is sensible)
3. tail risk:
 - `q90 - q50` (upper tail spread)
 - `worst - q50`
4. probability of "bad run":
Define a threshold, e.g. `objective > q75 + k*(q75-q25)` or `objective > baseline*(1+alpha)`
Compute fraction of seeds exceeding threshold.

Deliverable:

Risk profile table and per-model aggregates (mean risk across instances).

=====

PART VI - CONVERGENCE / LEARNING CURVES (TWO DIFFERENT THINGS)

=====

- A) Within-episode convergence (policy performance during an episode)
- B) Across-training learning curves (how the PPO improved during training)

Do both if you have logs.

Task 11A - Within-episode progress (test runs)

If you log `best_objective_over_time` within an episode (steps or time):

For each run (m,i,s) store series:
best(t) = best objective up to step t

Aggregate:

1. per (m,i): mean over 20 seeds, plus bands (min/max or q25/q75)
2. per model: aggregate across instances in the chosen evaluation set

Deliverable:

Within-episode progress plots and tables: "fast improvement vs late improvement".

Task 11B - Training learning curves (per model)

From PPO training logs extract over training timesteps:

1. mean episode return
2. value loss, policy loss, entropy
3. approx KL, clip fraction
4. explained variance
5. gradient norms (if available)

Then:

- compare stability across the 5 models
- identify signs of collapse or overfitting

Deliverable:

Training diagnostics report per model + cross-model comparison summary.

=====

PART VII - POLICY BEHAVIOR DIAGNOSTICS (OPERATOR/ACTION ANALOG)

=====

Interpret "operator usage" as "action usage" and "policy decisions".

Task 12 - Action usage distribution on test

For each run (m,i,s):

1. count of each discrete action, or summary stats for continuous actions
2. action entropy over time (if available)
3. fraction of invalid actions (if your env can produce them)
4. fraction of no-op actions (if defined)

Aggregate:

- per (m,i): mean over seeds
- per model: mean over instances

Deliverable:

Action usage tables and heatmaps (instances x actions).

Task 13 - Action effectiveness

Define an event-level delta similar to ALNS:

For step t in a run:

delta_t = objective_after_step_t - objective_before_step_t

For each action a:

1. $P(\text{improve} \mid \text{action}=a) = \text{fraction of steps with } \text{delta}_t < 0 \text{ when action}=a$

2. mean delta | action=a
3. mean improving delta | action=a and delta_t < 0
4. mean worsening delta | action=a and delta_t > 0
5. acceptance is not relevant unless you have rejection mechanics; use termination or invalidation

Deliverable:

Per-action effectiveness table, per model, aggregated across test instances and seeds.

Task 14 - State-conditional behavior (optional but strong)

If you can bucket states by a small set of features (e.g. violation count, slack, objective)

1. Define state buckets B1..Bk.
2. For each bucket and action:
 - action probability
 - action effectiveness (as in Task 13)

Deliverable:

A "policy map" table: buckets x actions.

=====

PART VIII - OVERFITTING / GENERALIZATION CHECKS

=====

Task 15 - Train vs test separation audit

For each model:

1. Confirm the 3 test instances were never used in training rollouts.
2. Confirm normalization stats (if any) were not fit on test instances.
3. Confirm no leakage via instance ID features or caching.

Deliverable:

Leakage audit checklist and conclusions.

Task 16 - Sensitivity to instance distribution

Because each model sees a different training subset of 12:

1. For each instance j in {1..15} compute how often it appears in training across the 5 models.
2. Check whether "rarely trained" instances are systematically harder in test.
3. If you can, run all 5 models on all 15 instances (20 seeds each) to remove split effects.

Deliverable:

Instance-frequency table + recommendation on whether to standardize evaluation.

=====

PART IX - VISUALIZATIONS (MINIMUM SET)

=====

1. For each model: boxplots of objective on each of its 3 test instances (20 seeds each).
2. Cross-model matrix heatmap: mean objective (models x instances) on the common evaluation set.
3. Pairwise scatter plots: model A vs model B per instance (means).
4. Risk plots: std and tail metrics per (model, instance).
5. Action usage heatmap (instances x actions) per model.
6. Action effectiveness bar charts per model.

=====

PART X - MINIMAL DEFENSIBLE CORE (IF YOU SKIP OPTIONALS)

=====

Do exactly:

1. Build the 300-row test-run dataset (Task 2) and the per-(model,instance) 20-seed summary
2. Report within-model held-out aggregate scores (Task 6A) including worst-case among the 3
3. Report seed variance and tail risk per (model,instance) (Task 10).
4. If you can re-run to create a common evaluation set: do Task 6B and then pairwise Wilcox

This is coherent for 5 PP0 models trained on different 12/3 splits with 20-seed test evalua